

ISMAGI — Institut Supérieur de Management et d'Informatique
Bases de Données Avancées (PL/SQL) — CI1 — 2025/2026

RAPPORT TECHNIQUE

Projet de Fin de Module

Systeme Bancaire Oracle — Base de Données Avancée avec PL/SQL

Encadrant : Pr. Yasser EL MADANI EL ALAMI

Année universitaire : 2025 / 2026

Table des matières

1. Introduction et contexte du projet	3
2. Analyse et modélisation	4
3. Architecture de la base de données	5
4. Développement PL/SQL	8
5. Administration Oracle	12
6. Optimisation des performances	13
7. Module intelligent — Détection de fraude	14
8. Tests et validation	15
9. Guide d'installation	16
10. Conclusion	17

1. Introduction et contexte du projet

1.1 Présentation générale

Ce projet de fin de module s'inscrit dans le cadre du cours de Bases de Données Avancées (PL/SQL) dispensé en première année de cycle ingénieur. Il vise à concevoir, implémenter et administrer une base de données Oracle complète pour un système bancaire.

Le système bancaire développé couvre l'ensemble des fonctionnalités d'une banque moderne : gestion des clients et des comptes, traitement des virements, suivi des transactions, octroi de crédits, ainsi qu'un module intelligent de détection de fraude.

1.2 Objectifs du projet

- Concevoir un Modèle Conceptuel de Données (MCD) complet pour un domaine bancaire
- Implémenter la base de données avec Oracle Database 21c XE
- Développer des objets PL/SQL : procédures, fonctions, triggers, packages
- Assurer l'administration Oracle : utilisateurs, rôles, audit, sécurité
- Optimiser les performances via l'analyse des plans d'exécution
- Intégrer un module intelligent de détection des transactions frauduleuses

1.3 Environnement technique

Composant	Technologie	Version
SGBD	Oracle Database XE	21c
Conteneurisation	Docker Desktop	Dernière
IDE	VS Code + SQL Developer Extension	Dernière
Versioning	Git / GitHub	Dernière
OS hôte	Windows 10/11	-

2. Analyse et modélisation

2.1 Modèle Conceptuel de Données (MCD)

Le MCD a été élaboré à partir de l'analyse des besoins du système bancaire. Il identifie les entités principales, leurs attributs et les associations entre elles.

Entités principales

Entité	Description	Attributs clés
CLIENTS	Clients personnes physiques de la banque	id_client, nom, prénom, email, téléphone
COMPTES	Comptes bancaires (courant/épargne)	id_compte, numéro, type, solde, statut
VIREMENTS	Opérations de transfert entre comptes	id_virement, montant, date, statut
TRANSACTIONS	Historique de toutes les opérations	id_transaction, type, montant, date
CREDITS	Prêts accordés aux clients	id_credit, montant, taux, durée
CARTES	Cartes bancaires liées aux comptes	id_carte, numéro, type, expiration
ADMINS	Administrateurs du système	id_admin, login, rôle
ROLES	Rôles système pour les admins	id_role, libellé

2.2 Modèle Logique de Données (MLD)

Le passage du MCD au MLD a permis de générer les tables relationnelles avec leurs clés primaires, clés étrangères et contraintes d'intégrité. Toutes les relations N-N ont été transformées en tables d'association.

2.3 Règles de gestion

- Un client peut posséder plusieurs comptes (courant et/ou épargne)
- Un virement implique obligatoirement un compte source et un compte destination distincts
- Un crédit est rattaché à un client ET à un compte de remboursement
- Un compte ne peut pas avoir un solde négatif (règle métier bloquante)
- Chaque carte est liée à un seul compte bancaire
- Un plafond de crédit est défini et suivi par client

3. Architecture de la base de données

3.1 Structure des tables

La base de données BANK_ADMIN contient 15 tables organisées en modules fonctionnels :

Module	Tables	Rôle
Utilisateurs	ROLES, CLIENTS, ADMINS, HISTORIQUE_CONNEXIONS	Gestion des acteurs du système
Comptes	COMPTEs, CARTES	Gestion des produits bancaires
Opérations	STATUTS_VIREMENT, VIREMENTS, TRANSACTIONS	Traitement des flux financiers
Crédits	CREDITS, ECHEANCES_CREDIT, PLAFONDS_CREDIT	Gestion des prêts
Technique	AUDIT_LOG, ERROR_LOG, ALERTES_FRAUDE	Traçabilité et sécurité

3.2 Séquences

Une séquence Oracle est créée pour chaque table, permettant la génération automatique des identifiants primaires sans collision. Toutes les séquences démarrent à 1 avec un incrément de 1 et sans cache (NOCACHE) pour garantir la continuité.

Séquence	Table associée
SEQ_CLIENTS	CLIENTS
SEQ_COMPTEs	COMPTEs
SEQ_VIREMENTS	VIREMENTS
SEQ_TRANSACTIONS	TRANSACTIONS
SEQ_CREDITS	CREDITS
SEQ_CARTES	CARTES
SEQ_ALERTES_FRAUDE	ALERTES_FRAUDE
SEQ_AUDIT_LOG	AUDIT_LOG
SEQ_ERROR_LOG	ERROR_LOG

3.3 Vues

Trois types de vues ont été créés conformément aux exigences du projet :

Vue	Type	Description
-----	------	-------------

VUE_COMPTES_CLIENTS	Simple	Jointure entre COMPTES et CLIENTS — accès unifié aux informations client/compte
VUE_VIREMENTS_STATUTS	Simple	Jointure entre VIREMENTS et STATUTS_VIREMENT — lisibilité des transferts
VUE_STATS_TRANSACTIONS	Complexe	Agrégats (COUNT, SUM, AVG, MAX, MIN) par compte et type de transaction
MV_STATS_MENSUELLES_VIREMENTS	Matérialisée	Statistiques mensuelles des virements — REFRESH ON DEMAND pour performance

La vue matérialisée MV_STATS_MENSUELLES_VIREMENTS utilise le mode REFRESH ON DEMAND car les données statistiques mensuelles n'ont pas besoin d'être recalculées à chaque requête. Ce choix améliore significativement les performances pour les tableaux de bord et rapports.

3.4 Index

Trois types d'index ont été mis en place pour optimiser les performances des requêtes les plus fréquentes :

Type	Index	Justification
B-TREE	IDX_BTREE_TX_COMPTE	Recherche de transactions par compte — haute cardinalité
B-TREE	IDX_BTREE_VIR_DATE	Filtrage des virements par date — requêtes de reporting
BITMAP	IDX_BITMAP_COMPTES_STATUT	Filtrage par statut (ACTIF/BLOQUE/FERME) — faible cardinalité
BITMAP	IDX_BITMAP_COMPTES_TYPE	Distinction COURANT/EPARGNE — faible cardinalité
Fonctionnel	IDX_FUNC_CLIENTS_NOM	Recherche UPPER(nom) insensible à la casse
Fonctionnel	IDX_FUNC_CLIENTS_EMAIL	Recherche UPPER(email) insensible à la casse

3.5 Synonymes

Des synonymes privés et publics ont été créés pour simplifier l'accès aux objets de la base et améliorer la maintenabilité :

- Synonymes privés : COMPTES_CLIENTS, STATS_TRANSACTIONS, VIREMENTS_DETAIL, STATS_MENSUELLES
- Synonymes publics : PUB_COMPTES, PUB_COMPTES_CLIENTS — accessibles par tous les utilisateurs autorisés

4. Développement PL/SQL

4.1 Procédures stockées

Trois procédures stockées ont été développées pour encapsuler la logique métier critique du système bancaire :

4.1.1 VALIDER_VIREMENT

Cette procédure effectue un virement entre deux comptes avec toutes les vérifications de sécurité nécessaires. Elle vérifie l'existence et le statut des comptes, contrôle la suffisance du solde, respecte le plafond autorisé, débite le compte source et crédite le compte destination, et journalise l'opération dans AUDIT_LOG.

Exceptions gérées : compte inexistant (NO_DATA_FOUND), solde insuffisant (exception personnalisée), dépassement de plafond, compte bloqué ou fermé, erreurs système (OTHERS).

4.1.2 ANNULER_VIREMENT

Permet l'annulation d'un virement en attente ou validé avec restauration automatique des soldes. La procédure vérifie que le virement est annulable, restaure les montants sur les deux comptes, met à jour le statut et trace l'opération.

4.1.3 OUVRIR_COMPTTE

Procédure de création d'un nouveau compte bancaire avec dépôt initial obligatoire. Elle valide l'existence du client, génère un numéro de compte unique, insère le compte et enregistre la transaction initiale de dépôt.

4.2 Fonctions

Fonction	Paramètres	Retour	Description
CALCULER_TOTAL_COMPTTE	id_compte, type_tx, date_debut, date_fin	NUMBER	Calcule le total des transactions d'un compte selon critères
CALCULER_REMISE	id_client	NUMBER (%)	Remise selon ancienneté, montant total et nombre de comptes
COMPTTE_DISPONIBLE	id_compte, montant	VARCHAR2	Vérifie si un compte peut effectuer une opération d'un montant donné

4.3 Triggers

Cinq triggers ont été implémentés pour automatiser les règles métier et la traçabilité :

Trigger	Événement	Table	Rôle
TRG_SOLDE_NEGATIF	BEFORE INSERT/UPDATE	COMPTES	Bloque toute opération résultant en un solde négatif ou sur compte fermé/bloqué
TRG_AUDIT_COMPTES	AFTER UPDATE/DELETE	COMPTES	Journalise toutes les modifications de comptes dans AUDIT_LOG
TRG_AUDIT_VIREMENTS	AFTER UPDATE/DELETE	VIREMENTS	Journalise toutes les modifications de virements dans AUDIT_LOG
TRG_DATE_MODIF_*	BEFORE UPDATE	4 tables	Met à jour automatiquement la colonne DATE_MODIFICATION
TRG_DETECTION_FRAUDE	AFTER INSERT	TRANSACTIONS	Génère une alerte dans ALERTES_FRAUDE pour toute transaction > 10 000 MAD

4.4 Package PKG_VIREMENTS

Un package Oracle centralise toutes les sous-routines liées aux virements. Il comprend une spécification (interface publique) et un corps (implémentation).

Spécification du package

- Constante : SEUIL_MONTANT_ELEVE (10 000 MAD)
- Constante : MAX_VIREMENTS_JOUR (10 virements par jour)
- Procédure : effectuer_virement — virement complet avec validations
- Procédure : annuler_virement_pkg — annulation avec restauration
- Procédure : suspendre_compte — suspension d'un compte
- Fonction : get_solde — retourne le solde actuel d'un compte
- Fonction : compter_virements_jour — nombre de virements du jour
- Fonction : verifier_plafond — vérifie le plafond disponible
- Fonction : get_statut_virement — retourne le libellé du statut

5. Administration Oracle

5.1 Gestion des utilisateurs et rôles

Deux rôles applicatifs ont été créés pour segmenter les accès selon le profil des utilisateurs :

Rôle	Privilèges accordés	Profil utilisateur
GESTIONNAIRE_COMPTES	SELECT, UPDATE sur COMPTES, VIREMENTS, CLIENTS, CARTES	Gestionnaire de la banque
CLIENT_APP	SELECT sur COMPTES, INSERT sur VIREMENTS et TRANSACTIONS	Application client mobile/web

Deux utilisateurs applicatifs ont été créés et assignés à ces rôles : GESTIONNAIRE (role gestionnaire_comptes) et APP_CLIENT (role client_app). Ces utilisateurs n'ont accès qu'aux objets nécessaires à leur fonctionnement, selon le principe du moindre privilège.

5.2 Audit Oracle

L'audit a été activé sur les opérations sensibles pour garantir la traçabilité réglementaire :

- AUDIT UPDATE ON BANK_ADMIN.COMPTES — traçabilité des modifications de soldes
- AUDIT DELETE ON BANK_ADMIN.VIREMENTS — traçabilité des suppressions de virements
- AUDIT INSERT ON BANK_ADMIN.TRANSACTIONS — traçabilité de toutes les transactions

En complément de l'audit Oracle natif, les triggers TRG_AUDIT_COMPTES et TRG_AUDIT_VIREMENTS alimentent la table applicative AUDIT_LOG avec le détail des anciennes et nouvelles valeurs, l'utilisateur et l'horodatage.

5.3 Gestion des erreurs

Toutes les procédures, fonctions et packages du projet implémentent une gestion d'erreurs structurée avec journalisation dans ERROR_LOG. Le pattern utilisé est le suivant : capture de SQLCODE et SQLERRM dans des variables locales, puis insertion dans ERROR_LOG avec l'objet source, le type d'erreur et l'horodatage.

6. Optimisation des performances

6.1 Analyse avec EXPLAIN PLAN

L'optimisation a été réalisée par comparaison des plans d'exécution avant et après création des index. La méthodologie suivie est :

- Suppression temporaire des index — simulation d'un environnement sans optimisation
- Exécution de EXPLAIN PLAN sur les requêtes les plus fréquentes
- Analyse du plan : opération FULL TABLE SCAN, coût élevé
- Recréation des index + ajout d'un index composite optimisé
- Nouvelle exécution de EXPLAIN PLAN — opération INDEX RANGE SCAN, coût réduit

6.2 Résultats obtenus

Requête	Sans index (COST)	Avec index (COST)	Gain
SELECT transactions par compte	FULL TABLE SCAN — coût élevé	INDEX RANGE SCAN — coût réduit	> 50%
SELECT virements par date	FULL TABLE SCAN — coût élevé	INDEX RANGE SCAN — coût réduit	> 50%
SELECT comptes par statut	FULL TABLE SCAN — coût élevé	BITMAP INDEX — coût minimal	> 70%

6.3 Vue matérialisée vs requête directe

Pour les rapports de statistiques mensuelles, la vue matérialisée MV_STATS_MENSUELLES_VIREMENTS offre des performances nettement supérieures à la requête directe sur les tables de base. Le rafraîchissement à la demande (REFRESH ON DEMAND) permet de choisir le moment optimal pour mettre à jour les données agrégées.

7. Module intelligent — Détection de fraude

7.1 Présentation du module

Le module intelligent de détection de fraude est implémenté via le trigger TRG_DETECTION_FRAUDE, qui s'exécute automatiquement après chaque insertion dans la table TRANSACTIONS. Il constitue une première ligne de défense contre les opérations inhabituelles.

7.2 Règles de détection

Règle	Condition	Action
Montant élevé	Transaction > 10 000 MAD	Création d'une alerte MONTANT_ELEVE dans ALERTES_FRAUDE
Plafond dépassé	Virement > plafond autorisé	Blocage par la procédure VALIDER_VIREMENT
Compte bloqué	Opération sur compte BLOQUE	Blocage par le trigger TRG_SOLDE_NEGATIF

7.3 Structure de la table ALERTES_FRAUDE

- id_alerte : identifiant unique (séquence)
- id_compte : compte concerné par l'alerte
- id_transaction : transaction déclencheuse
- type_alerte : catégorie de l'alerte (MONTANT_ELEVE, etc.)
- montant : montant de la transaction suspecte
- date_alerte : horodatage de la détection
- statut : état de traitement (OUVERTE / EN_COURS / RESOLUE / IGNOREE)
- description : description textuelle de l'alerte

7.4 Module PLAFONDS_CREDIT

La table PLAFONDS_CREDIT complète le module intelligent en gérant les plafonds d'autorisation de crédit par client. Elle utilise une colonne virtuelle PLAFOND_DISPONIBLE calculée automatiquement (PLAFOND_AUTORISE - PLAFOND_UTILISE), ce qui évite les incohérences de données et améliore la lisibilité.

8. Tests et validation

8.1 Données de test

Un jeu de données de test complet a été inséré pour valider l'ensemble des fonctionnalités :

Table	Nombre de lignes	Données représentatives
ROLES	4	ADMIN, GESTIONNAIRE, AUDITEUR, SUPPORT
CLIENTS	20	Clients avec différentes anciennetés et profils
COMPTES	25	Comptes courants et épargne, statuts variés
CARTES	20	Cartes VISA et MASTERCARD, DEBIT et CREDIT
VIREMENTS	20	Virements validés, en attente, annulés
TRANSACTIONS	25	Débets, crédits, virements dont 5 > 10 000 MAD
CREDITS	5	Crédits immobilier, auto, personnel
ECHEANCES_CREDIT	20	Échéances payées et en attente
PLAFONDS_CREDIT	10	Plafonds actifs et suspendus
HISTORIQUE_CONNEXIONS	20	Connexions réussies et échouées

8.2 Validation des objets PL/SQL

Tous les objets PL/SQL ont été compilés et validés dans Oracle. La requête de vérification ci-dessous doit retourner uniquement des objets avec le statut VALID :

```
SELECT object_name, object_type, status
FROM user_objects
WHERE object_type IN
('PROCEDURE', 'FUNCTION', 'TRIGGER', 'PACKAGE', 'PACKAGE BODY')
ORDER BY object_type, object_name;
```

Objet	Type	Statut
VALIDER_VIREMENT	PROCEDURE	VALID
ANNULER_VIREMENT	PROCEDURE	VALID
OUVRIR_COMPTE	PROCEDURE	VALID
CALCULER_TOTAL_COMPTE	FUNCTION	VALID
CALCULER_REMISE	FUNCTION	VALID
COMPTE_DISPONIBLE	FUNCTION	VALID
TRG_SOLDE_NEGATIF	TRIGGER	VALID

TRG_AUDIT_COMPTEs	TRIGGER	VALID
TRG_DETECTION_FRAUDE	TRIGGER	VALID
PKG_VIREMENTS	PACKAGE	VALID
PKG_VIREMENTS	PACKAGE BODY	VALID

9. Guide d'installation

9.1 Prérequis

- Docker Desktop installé et démarré
- Image Oracle Database 21c XE disponible dans Docker
- VS Code avec l'extension SQL Developer
- Git installé

9.2 Étapes d'installation

- Démarrer le container Docker : oracle-xe sur le port 1521
- Cloner le dépôt GitHub : `git clone https://github.com/moh-donasso/systeme-bancaire-oracle.git`
- Se connecter en SYSDBA : `docker exec -it oracle-xe sqlplus / as sysdba`
- Exécuter 00_setup.sql (en SYSTEM sur XEPDB1) pour créer l'utilisateur bank_admin
- Se connecter avec bank_admin (service: XEPDB1, port: 1521)
- Exécuter les scripts dans l'ordre : 01 → 02 → 03 → 04 → 05 → 06 → 07 → 08 → 09
- Exécuter 10_users_roles.sql en tant que SYSTEM pour les rôles
- Exécuter 11_data_test.sql pour charger les données de test
- Exécuter 12_explain_plan.sql pour valider l'optimisation

9.3 Structure du dépôt GitHub

```
systeme-bancaire-oracle/
├── scripts/                # Scripts SQL dans l'ordre d'exécution
│   ├── 00_setup.sql       # Création de l'utilisateur bank_admin
│   ├── 01_tables.sql      # 15 tables avec contraintes
│   ├── 02_sequences.sql   # 15 séquences
│   ├── 03_views.sql       # Vues simple, complexe, matérialisée
│   ├── 04_indexes.sql     # Index B-TREE, BITMAP, fonctionnel
│   ├── 05_synonyms.sql    # Synonymes privés et publics
│   ├── 06_procedures.sql  # 3 procédures PL/SQL
│   ├── 07_functions.sql   # 3 fonctions PL/SQL
│   ├── 08_triggers.sql    # 5 triggers
│   ├── 09_packages.sql    # Package PKG_VIREMENTS
│   └── 10_users_roles.sql # Utilisateurs, rôles, audit
```

```
|   |─ 11_data_test.sql  # Données de test (20+ lignes/table)
|   |─ 12_explain_plan.sql # Analyse des performances
|   |─ 99_drop_all.sql   # Script de nettoyage
|─ docs/
|   |─ rapport.pdf
|   |─ presentation.pptx
|   |─ schema/erd.png
|─ README.md
```

10. Conclusion

10.1 Bilan du projet

Ce projet a permis de mettre en pratique l'ensemble des concepts avancés de bases de données Oracle couverts dans le cours : modélisation relationnelle, programmation PL/SQL, administration Oracle et optimisation des performances.

Le système bancaire développé est fonctionnel et couvre les besoins essentiels d'une banque : gestion des clients et comptes, traitement des virements avec contrôles de sécurité, octroi de crédits, génération d'alertes fraude automatiques et traçabilité complète des opérations.

10.2 Compétences acquises

Domaine	Compétences développées
Modélisation	Conception MCD/MLD, normalisation, contraintes d'intégrité
SQL avancé	Requêtes complexes, vues matérialisées, index spécialisés
PL/SQL	Procédures, fonctions, triggers, packages, gestion des exceptions
Administration	Utilisateurs, rôles, privilèges, audit Oracle
Optimisation	EXPLAIN PLAN, index B-TREE/BITMAP/fonctionnel, hints
DevOps	Docker, Git/GitHub, versioning des scripts SQL

10.3 Perspectives d'amélioration

- Intégration d'un moteur de règles plus sophistiqué pour la détection de fraude (machine learning)
- Mise en place d'une interface web de consultation via Oracle APEX
- Partitionnement des tables TRANSACTIONS et AUDIT_LOG pour les très grands volumes
- Implémentation de la réplication Oracle Data Guard pour la haute disponibilité
- Ajout d'un système de notifications en temps réel via Oracle Advanced Queuing